

Virtualizare procesare.

Configurare Cluster de Calculatoare

Obiectivul lucrării

Lucrarea își propune implementarea unui cluster de calculatoare și studierea modului în care calculatoarele componente interacționează în rețeaua locală de calculatoare.

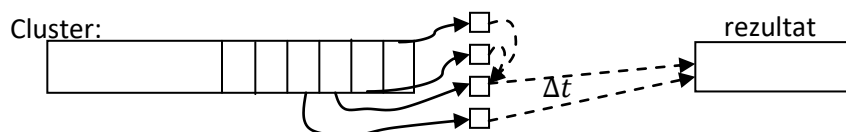
Breviar teoretic

Un cluster de calculatoare este un grup de calculatoare interconectate printr-o rețea de calculatoare, care lucrează împreună, astfel încât, în multe privințe acestea formează un singur calculator.

Clusterelor sunt, în general, conectate printr-o rețea LAN (Local Area Network) pentru ca viteza și latența să fie reduse. Suplimentar, în funcție de modul de funcționare, securitatea interconexiunilor se pretează pentru o rețea locală privată, care nu expune public toată infrastructura ci decât anumite funcții.

Programe paralele care rulează pe unul dintre nodurile clusterului utilizează puterea de procesare a tuturor nodurilor și produce rezultatul.

a) Cluster



Datorită evoluției performanțelor procesoarelor s-a dezvoltat o ramură specializată în procesarea de mare viteză folosind ca mediu de transfer între calculatoare rețelele locale. Acestea au ajuns la un nivel în care oferă lățime de bandă mare și latență mică;

- Calculatoarele dintr-un cluster funcționează împreună ca un singur calculator;
- Există mai multe tipuri de cluster:
 - o High availability cluster (HA) → au scopul de a îmbunătăți disponibilitatea serviciilor oferite;
 - o Load balancing cluster (LB) → apare când mai multe calculatoare împart sarcinile de lucru funcționând ca un singur calculator virtual;
 - o Cluster pentru calcul (CC) → rolul lor este de a efectua calcule;
→ poate necesita comunicații mai frecvente între noduri decât alte cluster;

În informatică FLOPS este un acronim ce provine de la expresia engleză floating point operations per second (tradus: operații în virgulă mobilă pe secundă). FLOPS reprezintă o unitate de măsură a puterii de calcul a unui calculator sau sistem de calcul, măsurând numărul maxim de operații în virgulă mobilă, (de regulă adunări și înmulțiri), ce sunt executate pe secundă. Unitatea FLOPS își găsește folosul mai ales în domeniul calculelor științifice, la care calculul în virgulă mobilă este frecvent folosit. estimativ: 2010, the fastest six-core PC processor reaches 109 gigaFLOPS (Intel Core i7 980 XE; Nvidia Tesla C2050 GPU computing processors perform around 515 gigaFLOPS).

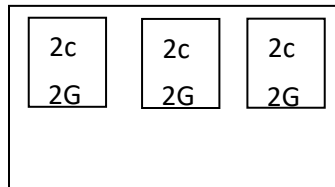
Un cluster este un grup de calculatoare functionand unitar si care se comporta in multe aspecte ca un singur calculator. In majoritatea cazurilor clusterelor sunt conectate la rețele de calculatoare.

Cele trei tipuri de cluster sunt:

- High availability cluster (HA) – stau online mereu;
- Load balancing cluster (LB) – rol de a balansa incarcarea echipamentelor;
- HPC Cluster pentru calcul (pentru procesare).

Clusterelor de tipul SSI rulează un SO unitar care repartizează sarcinile prin intermediul funcțiilor specifice pe diferite noduri.

SSI cluster:



6 core-uri

6G memorie

Comportamentul este apropiat de cel al SO distribuite, insa functiile oferite sunt mai restranse fata de un SO distribuit.

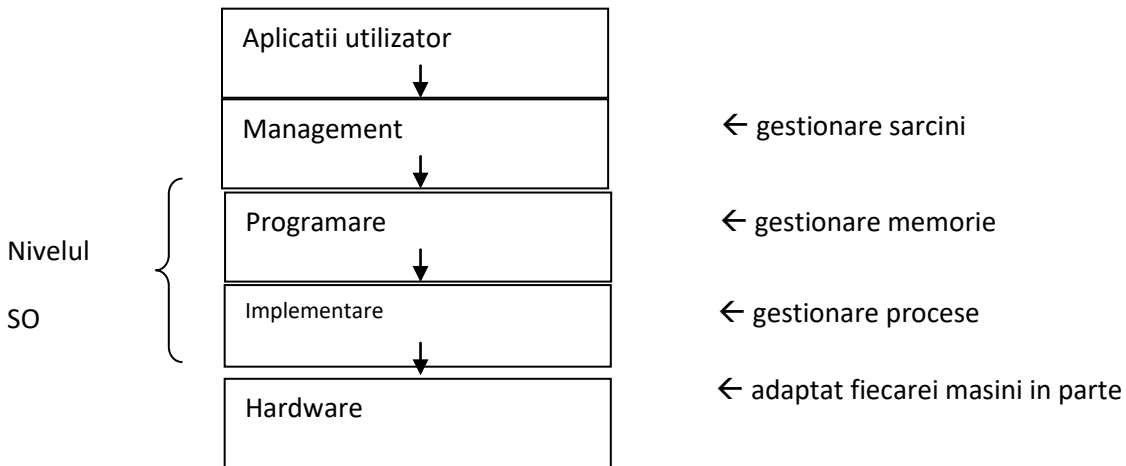
Principala activitate gestionata este migrarea proceselor si gestoranrea resurselor.

Avantajele clusterelor SSI:

- Punct unic de intrare pentru utilizatorii sau procesele care se conecteaza la cluster;
- Interfata utilizator unica (oferind inclusiv interfata grafica);
- Spatiu unic de adrese pentru procese (implica identificatori unici la nivelul clusterului pentru procese simplificand comunicatia intre acestia);
- Spatiu unic de intrare-iesire;
- Sistem unic de fisiere (deobicei NFS);
- Gestionarea unitara;

- Salvarea unitara a starilor intermediare.

Se disting urmatoarele niveluri in arhitectura unui cluster:



1. Nivelul hardware preia resursele fiecarui calculator si le pune la dispozitia SO al clusterului.
 - Transmisia intre noduri poate fi broadcast sau unicast;
 - Se asigura mecanisme pentru detectia erorilor in situatia in care anumite statii devin indisponibile.
2. Nivelul SO
 - Scopul este paralelizarea aplicatiilor secventiale;
 - Rolul este planificarea proceselor, identificarea resurselor libere, migrarea proceselor si balansarea incarcarii (load balancing). Accesarea acestor functii se face fara comenzi suplimentare.
3. Nivelul middleware (management)
 - Se interpune intre SO si aplicatiile utilizator cu rol de management al sarcinilor care sunt executate;
 - Planificarea globala a sarcinilor ofera un nivel ridicat de transparenta utilizatorului.
4. Nivelul aplicatie
 - Este nivelul ce ruleaza aplicatiile utilizator si interfata grafica (GUI) a SO.

In prezent se intocmesc ierarhii cu cele mai performante calculatoare virtuale; ordinul de marime privind performanta este de peta flops (peta= 10^{15} operatii in virgula mobila).

Exemple de cluster: Beowulf. Cateva din distributiile de linux care implementeaza acest tip de cluster: MOSIX, geared toward computationally intensive, IO-low, applications; ClusterKnoppix, based on Knoppix; Kerrighed.

Caracteristici sisteme cluster

- Sisteme strans cuplate (Tightly coupled systems);
- Creaza impresia unui singur sistem de calcul (Single system image);
- Sistem centralizat de Job management si planificare

În general, grupurile sunt strâns cuplate de exemplu. Toate placile de baza vor fi stivuite într-un singur dulap și conectate cu ajutorul unor rețea de interconectare. Vor împărtăși RAM, hard disk si alte periferice. Sistem de operare ruleaza pe unul dintre nodurile și controlează activitățile altor noduri.

Clusterelor Beowulf sunt grupuri ieftine create folosind de pe componentele raft. Puteți crea un cluster Beowulf, cu doar câteva computere ieftine și un segment Ethernet în curte. Deși nu vă dau performanțe de top-notch, performanța lor este de multe ori mai bine decât un singur calculator. O variantă de Beowulf Clusterelor permite sistemului de operare să ruleze pe fiecare nod și să permită încă de procesare paralelă. Și aceasta este exact ceea ce vom face aici.

Aplicații

Instalare Cluster Beowulf

Instalam Ubuntu Server 18.04 pe minim 3 masini virtual create cu VirtualBox. Placa de retea pentru aceste masini va fi in modul bridge.

O masina va fi configurata drept nod central si celelalte doua drept noduri de procesare (clienti). **Este necesara selectarea OpenSSH in timpul instalarii Ubuntu Server cat si configurarea placii de retea in timpul instalarii** (implicit placa de retea nu este configurata!).

Dupa instalare este necesar sa testam pe fiecare masina conectivitatea la internet cat si sa realizam actualizarea referintelor sistemului de operare cu: **sudo apt-get update**

1. Login in fiecare masina cu user si parola.



2. Adaugam in fisierul "hosts" ip-urile pe toate masinile virtuale cu comanda:

```
sudo nano /etc/hosts

ubuntu [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
GNU nano 2.9.3 /etc/hosts

127.0.0.1 localhost
192.168.86.20 grendel-3
192.168.86.15 grendel-0
192.168.86.9 grendel-1
# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters

^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text   ^J Justify    ^C Cur Pos   M-U Undo
^X Exit      ^R Read File  ^M Replace   ^U Uncut Text ^T To Spell  ^G Go To Line M-E Redo

[ Read 10 lines ]
Right Ctrl
```

Acest fisier trebuie sa arate asa:

```
127.0.0.1 localhost
192.168.133.100 ub0
192.168.133.101 ub1
192.168.133.102 ub2
```

Nu asa:

```
127.0.0.1 localhost
127.0.1.1 ub0
192.168.133.100 ub0
192.168.133.101 ub1
192.168.133.102 ub2
```

Primul rand ramane neschimbat, al 2 lea rand reprezinta adresa serverului + nume server (adresa unei masini se poate afla tastand comanda "ip a", Aceste adrese sunt cele setate initial la instalare). Pe urmatoarele linii se pun adresele clientilor urmate de numele acestora. Ctrl+O pentru salvare Ctrl+X pentru iesire.

3. Instalam "nfs-server" pe server si "nfs-client" pe client (Rolul "nfs-ului" este de a partaja un fisier de pe master cu clientii, adica faci un fisier pe master si il vezi si pe clienti)

a. Comanda pentru nfs-server :

```
omid@ub0:~$ sudo apt-get install nfs-server
```

b. Comanda pentru nfs-client:

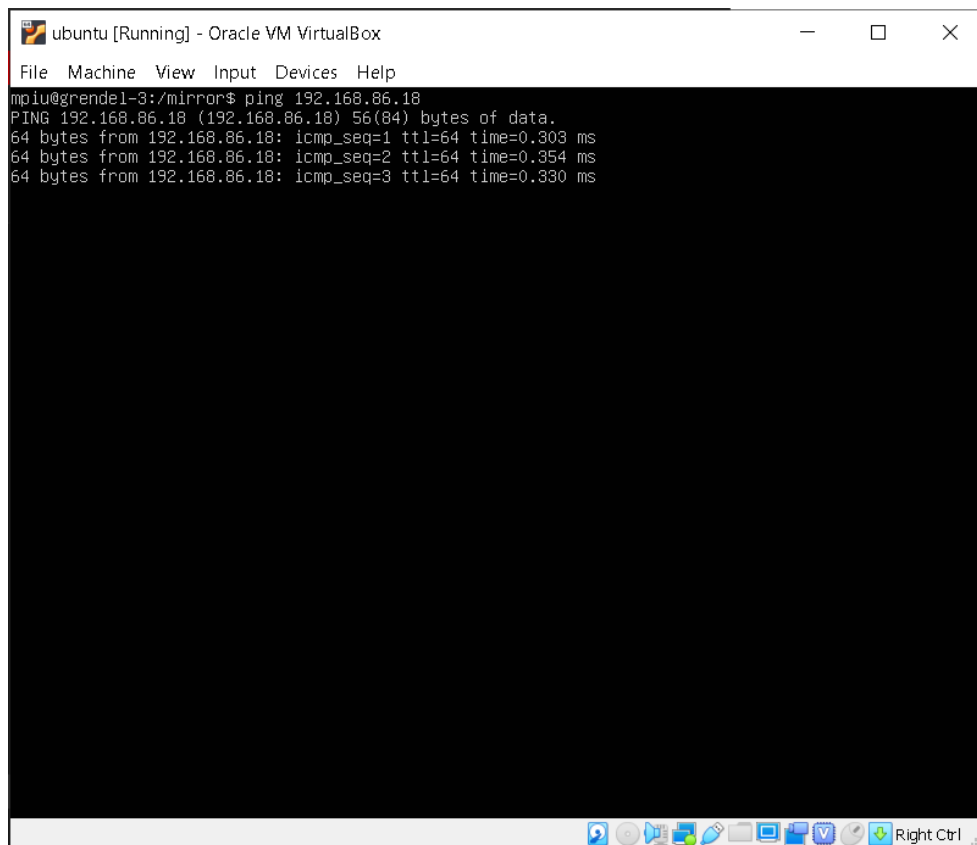
```
omid@ub1:~$ sudo apt-get install nfs-client
```

4. Pentru a verifica daca toate masinile virtuale au acces una la alta o sa folosim comanda ping sa verificam daca exista conexiune intre acestea (dati ping de la fiecare masina catre fiecare):

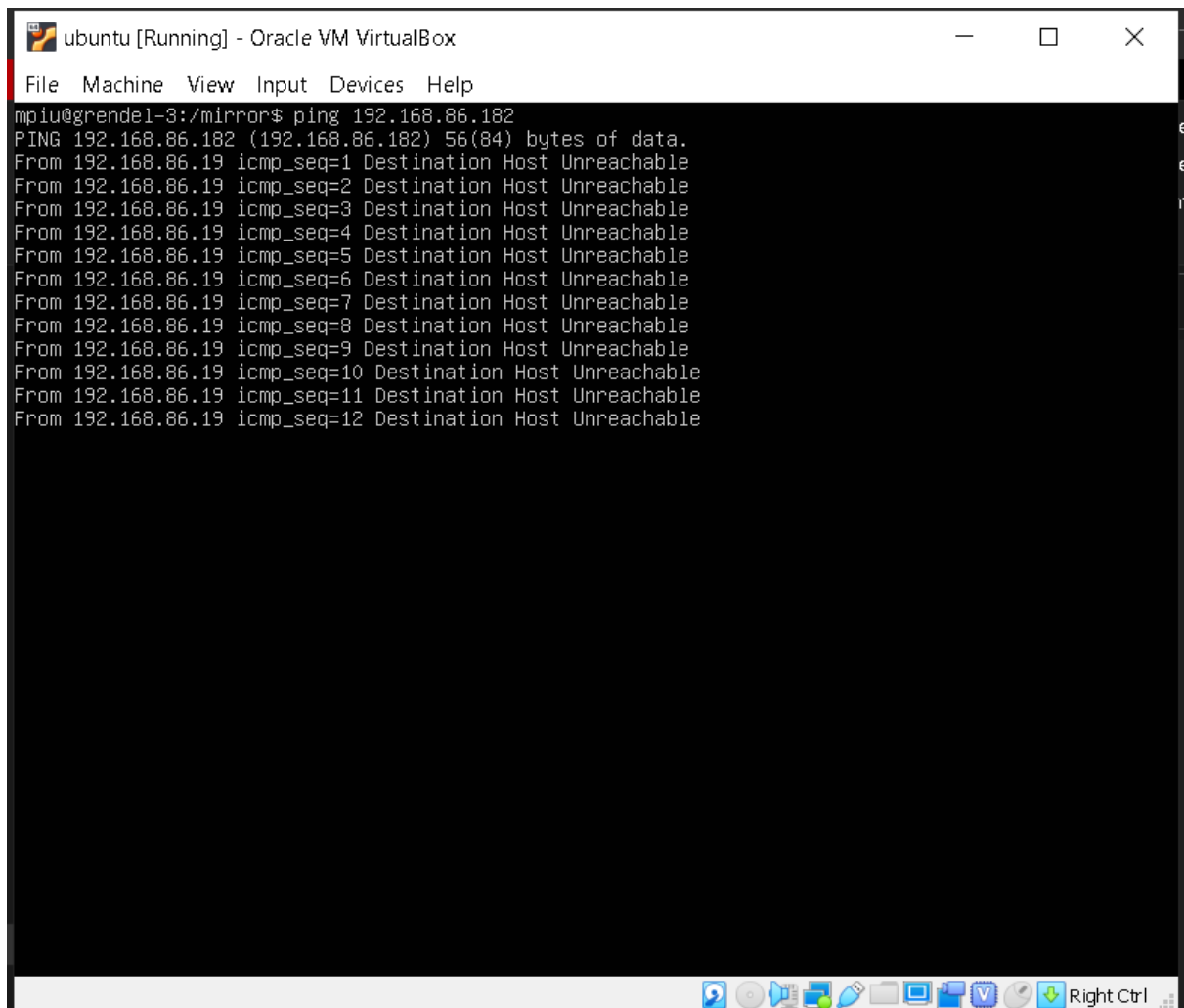
Exemplu (daca suntem pe server dam ping catre ub0):

```
omid@ub0:~$ ping 192.168.94.101 ( bagati ip-ul de la masinile  
voastre virtuale)
```

Asa trebuie sa arate:



Nu ca in imaginea de mai jos:



The screenshot shows a terminal window titled "ubuntu [Running] - Oracle VM VirtualBox". The terminal output shows a ping command being executed from the host "mpiu@grendel-3:/mirror" to the IP address "192.168.86.182". The output indicates that the destination host is unreachable for all 12 ping attempts. The terminal window has a menu bar with "File", "Machine", "View", "Input", "Devices", and "Help". The bottom of the window shows a taskbar with various icons and a "Right Ctrl" button.

```
mpiu@grendel-3:/mirror$ ping 192.168.86.182
PING 192.168.86.182 (192.168.86.182) 56(84) bytes of data.
From 192.168.86.19 icmp_seq=1 Destination Host Unreachable
From 192.168.86.19 icmp_seq=2 Destination Host Unreachable
From 192.168.86.19 icmp_seq=3 Destination Host Unreachable
From 192.168.86.19 icmp_seq=4 Destination Host Unreachable
From 192.168.86.19 icmp_seq=5 Destination Host Unreachable
From 192.168.86.19 icmp_seq=6 Destination Host Unreachable
From 192.168.86.19 icmp_seq=7 Destination Host Unreachable
From 192.168.86.19 icmp_seq=8 Destination Host Unreachable
From 192.168.86.19 icmp_seq=9 Destination Host Unreachable
From 192.168.86.19 icmp_seq=10 Destination Host Unreachable
From 192.168.86.19 icmp_seq=11 Destination Host Unreachable
From 192.168.86.19 icmp_seq=12 Destination Host Unreachable
```

5. Pe toate masinile virtuale facem folderul "mirror" folosind comanda

Comanda:

```
omid@ub0:~$ sudo mkdir /mirror
```

6. Pentru a partaja folderul cu clientii trebuie sa adaugam o comanda in folderul "/etc/exports".Comanda se aplica **doar pe server**.

Acest lucru se poate face cu urmatoarea comanda:

```
omid@ub0:~$ echo "/mirror *(rw, sync)" | sudo tee -a /etc/exports
```

Acum dam restart la nfs server cu urmatoarea comanda:

```
omid@ub0:~$ sudo service nfs-kernel-server restart
```

7. Din cand in cand mai dam un update pe fiecare masina cu comanda:

```
omid@ub0:~$ sudo apt-get update
```


8. Facem legatura intre fisiere de la client la server cu urmatoarele comenzi(fiecare comanda se executa pe masinile virtuale client):

```
omid@ub1:~$ sudo mount ub0:/mirror /mirror  
omid@ub2:~$ sudo mount ub0:/mirror /mirror  
(ub0 este numele serverului)
```

9. Acum, pentru a verifica daca totul functioneaza bine verificam daca folderul mirror este sincronizat (Avem un folder numit **mirror** atat pe **client** cat si pe **server**, daca cream un fisier in folderul miror pe **server** trebuie sa il vedem si in folderul mirror de pe **client**)
Pentru a face acest lucru rulam urmatoarele comenzi :
Pe **server** : `sudo touch FisierCreatPeServer` (aceasta comanda creeaza un fisier)
Pe **clienti** : `cd /mirror && ls` (Ne afiseaza ce se afla in folderul mirror)
In acest moment ar trebui sa ne afiseze numele fisierului creat anterior
("FisierCreatPeServer")

The screenshot shows a terminal window titled "ubuntu [Running] - Oracle VM VirtualBox". The terminal output is as follows:

```
mpiu@grendel-3:/mirror$ cd /mirror && ls  
machinefile  mpi_hello2.c  mpiu  test.c  
accestFisierEsteCreatPeServer  mpi_hello  mpi_hello.c  test  
mpiu@grendel-3:/mirror$ _
```

Blue boxes highlight the command `cd /mirror && ls` and the output `accestFisierEsteCreatPeServer`. A blue arrow points from the text "aceasta comanda creeaza un fisier" in the previous block to the command `accestFisierEsteCreatPeServer` in the terminal output.

10. Facem un user numit "mpiu" cu parola "mpiu" pe server care are directorul /home in folderul mirror

```
sudo adduser -home /mirror/mpiu mpiu
```

11. Schimbam ownerul folderului /mirror cu mpiu

```
omid@ub0:~$ sudo chown mpiu /mirror
```

12. In cazul in care nu ati bifat ssh la instalarea masinilor virtuale rulati comanda:

```
omid@ub0:~$ sudo apt-get install openssh-server
```

13. Adaugam userul mpiu la sudoers ruland comanda (va cere parola)

```
sudo adduser mpiu sudo
```

14. Ne conectam la userul mpiu (va cere parola)

```
omid@ub0:~$ su - mpiu
```

15. Rulati mpiu@ub0:~\$ ssh-keygen -t rsa

16. Pentru a fi accesat clientul fara parola rulam:

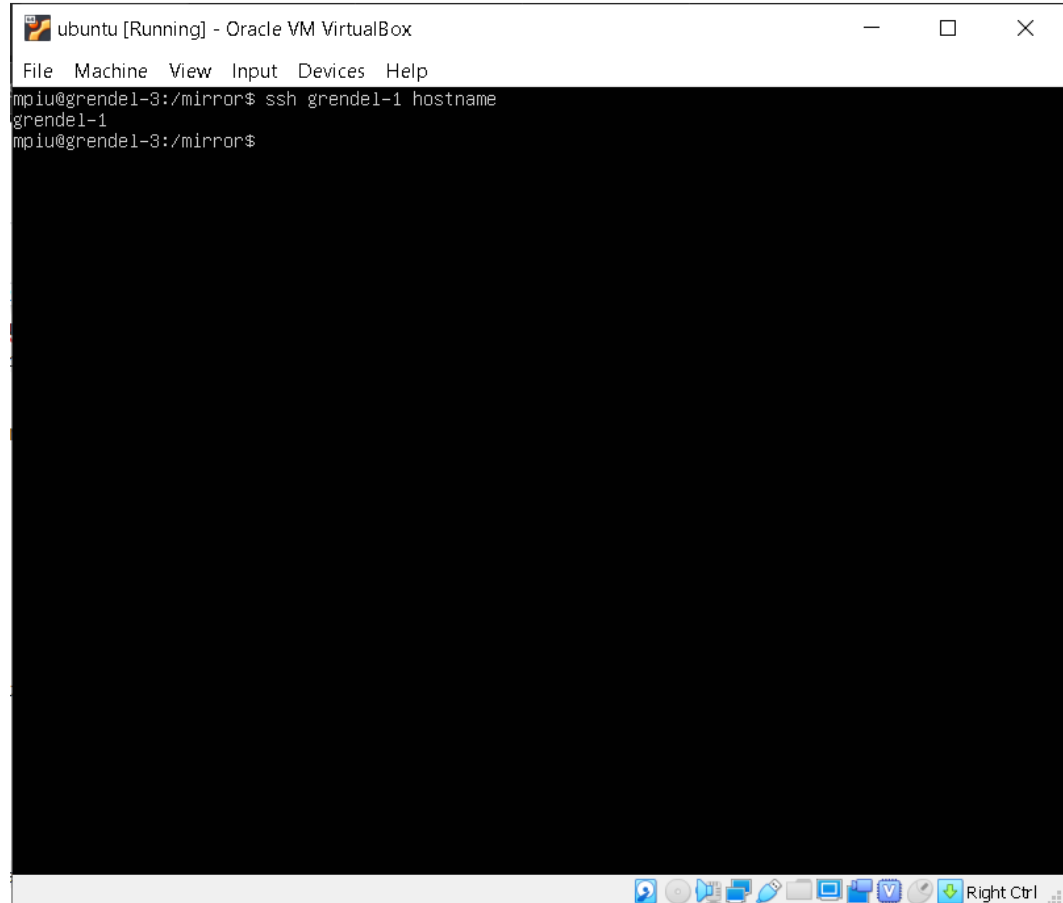
```
ssh-copy-id <hostname>
```

unde <hostname> = ip client (se ruleaza pentru fiecare client)

```
ex:ssh-copy-id 192.168.94.10
```

17. Testam functionalitatea ssh-ului ruland de pe server :

```
ssh ub1 hostname
```



Daca **nu ne cere** nicio parola atunci functioneaza **bine**. In cazul in care va cere parola trebuie sa reluati de la pasul 16

18. Instalam compilerul

```
mpiu@ub0:~$ sudo apt-get install build-essential
```

19. Instalam mpi

```
sudo apt-get install mpich2
```

20. Cream fisierul "machinefile" cu comanda :

```
sudo nano machinefile
```

in folderul mirror in care introducem:

```
ub3:4
```

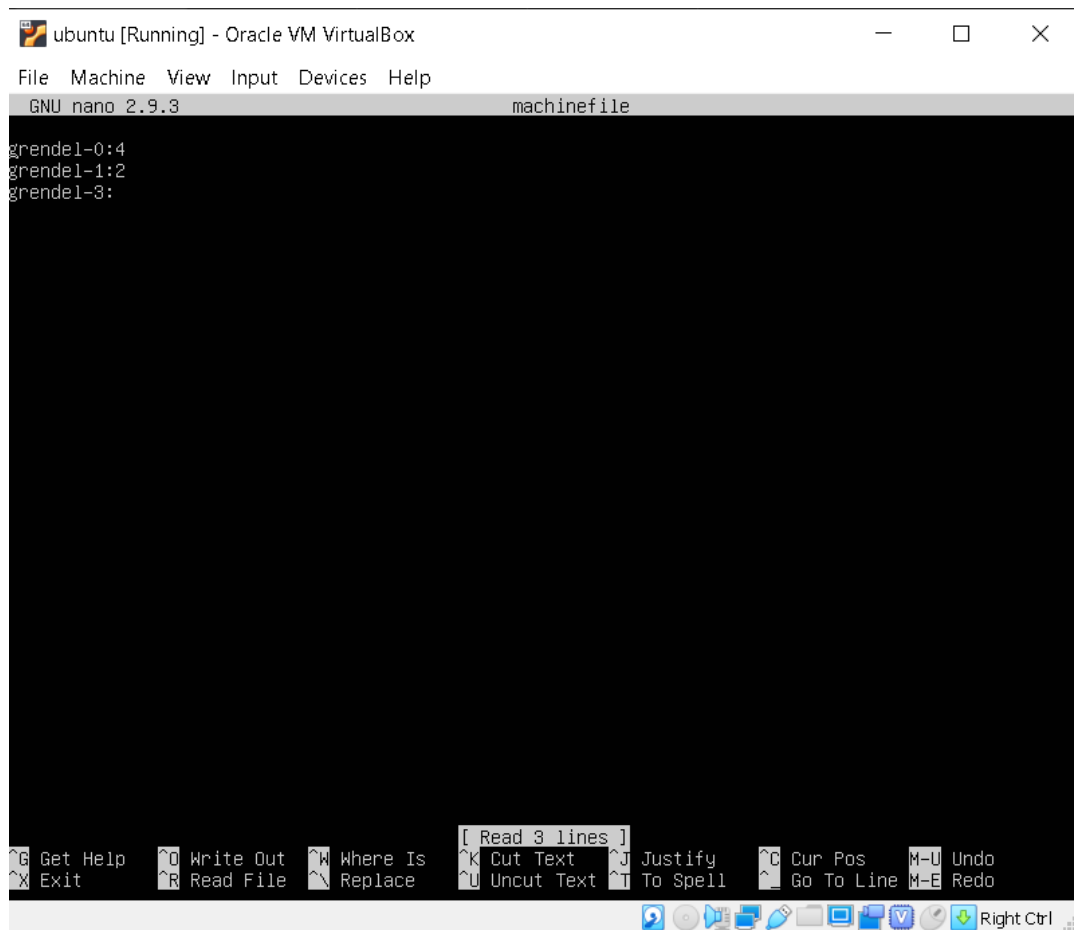
```
ub2:2
```

```
ub1
```

```
ub0
```

Ctrol-O pentru salvare

Ctrl-X prntru iesire



ubuntu [Running] - Oracle VM VirtualBox

File Machine View Input Devices Help

GNU nano 2.9.3 machinefile

```
grendel-0:4
grendel-1:2
grendel-3:
```

Get Help Write Out Where Is Cut Text Justify Cur Pos M-U Undo
Exit Read File Replace Uncut Text To Spell Go To Line M-E Redo

Right Ctrl

21. Construim programul care testeaza daca toate procesoarele functioneaza in paralel

In folderul mirror rulam comanda:

`sudo nano mpi_hello.c` , in care scriem programul de mai jos

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int myrank, nprocs;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &myrank);

    printf("Hello from processor %d of %d\n", myrank, nprocs);

    MPI_Finalize();
    return 0;
}
```

22. Compilam programul

`mpiu@ub0:~$ mpicc mpi_hello.c -o mpi_hello`

Dupa compilare ne va rezulta fisierul "mpi_hello" pe care il rulam la pasul urmator

23. Rulam rezultatul

```
mpiu@ub0:~$ mpiexec -n 8 -f machinefile ./mpi_hello
```

Rezultatul final:

```
Hello from processor 0 of 8  
Hello from processor 1 of 8  
Hello from processor 2 of 8  
Hello from processor 3 of 8  
Hello from processor 4 of 8  
Hello from processor 5 of 8  
Hello from processor 6 of 8  
Hello from processor 7 of 8
```